Box technikák felismerése gépi tanulással

Készítette: Rácz Elemér Roland

1. A projektről
2. Felhasznált eszközök
3. Videófelvétel
4. Koordináta felvétel
5. Adatok feldolgozása
6. Modell kitanítása
7. Tesztelés
8. Fejlesztési tervek

1. A projektről

A projekt végső célja egy mesterséges intelligencia modell kitanítása, mely képes kamera felvétel alapján felismerni és aktív visszajelzéssel kritizálni az alany box technikáit. A Haladó Programozás nevezetű tantárgy kereteiben egy kezdetleges próba programot hoztam létre, mely megismertette velem a szükséges eszközöket és a gépi látás alapjait, és szabály alapon képes volt felismerni ütéseket, de hanyagolható pontossággal.

Az Informatikai Projektfeladat nevezetű óra kereteiben, ezen projektet tovább fejlesztettem, az eddigi alapokra építve. Egy Keras modell került kitanításra, mely videófelvételeket fogad, és képes felismerni a felső test szögei alapján az ütés formákat, sokkal erősebb pontossággal.

2. Felhasznált eszközök

A videófelvételeket, melyeket mind kitanításhoz, mind teszteléshez felhasználtam, egy OBSBOT MEET 2 webkamerával vettem fel. Az összes felvétel 3840 x 2160 pixeles képfelbontással, és 30 FPS képfrissítéssel készült, garantálva a maximális tisztaságot a képfeldolgozás szempontjából, valamint a konzisztens másodpercenkénti képkockákat.

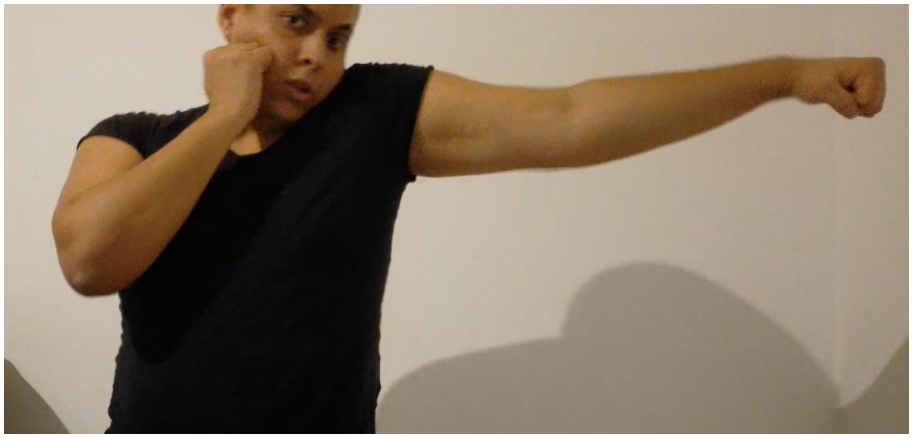
A program intenzív erőforrás igényeit saját számítógépemmel szolgáltattam, mely 24 GB memóriát, Intel(R) Core(TM) i7-8750H @ 2.20GHz (2.21 GHz) processzort valamint NVIDIA GeForce GTX 1660 Ti, 6 GB memóriával rendelkező videókártyát tartalmaz.

3. Videófelvétel

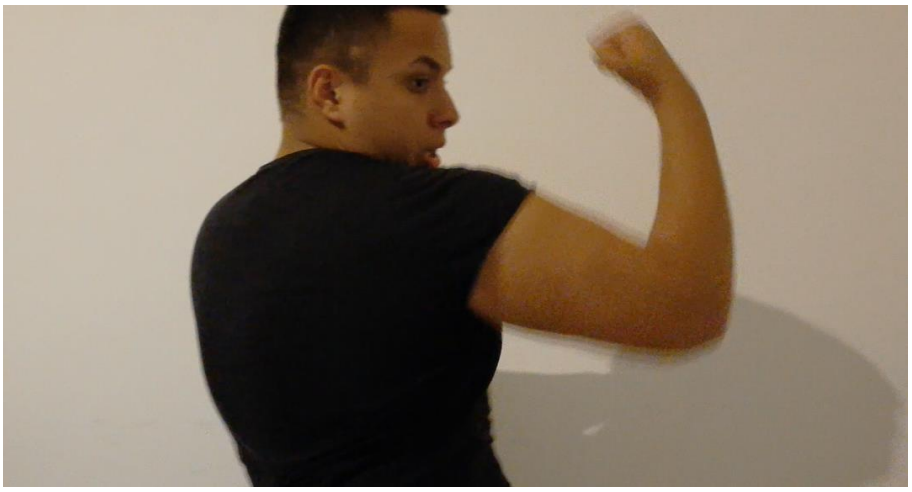
A pontos gépi tanulás érdekében, pontosan kellett determinálni a felvételek részeleteit, mielőtt bármilyen video készülne. Jelenlegi megoldás arra hagyatkozott, hogy a test fontosabb pontjai tisztán látszódnak a felvételeken, mivel ezek alapján végezzük a modell kitanítását. Ennek érdekében, felső testel a kamera felé fordulva, körülbelül 45 fokos szögben félprofillal készült az összes felvétel.

A box alapjait három ütésfajta adja. Az egyenes, ami a kar teljes kinyúlásával és váll intenzív előre tolásával jár. A horog, mely fontosabb komponensei az előre tolt váll, a körülbelül 90 fokos szög a könyöknél, valamint az erős csípőforgatás, ami a mozdulat vízszintes tengelyű pályáját adja. Végül, a felütés, ami hasonló a horoghoz, de a mozdulat függőleges erőkifejtést igényel, mintha 90 fokkal el lenne forgatva a mozdulat. Ezen ütéseket mindkét karral elvégeztem, tisztán látható szögből, minimális

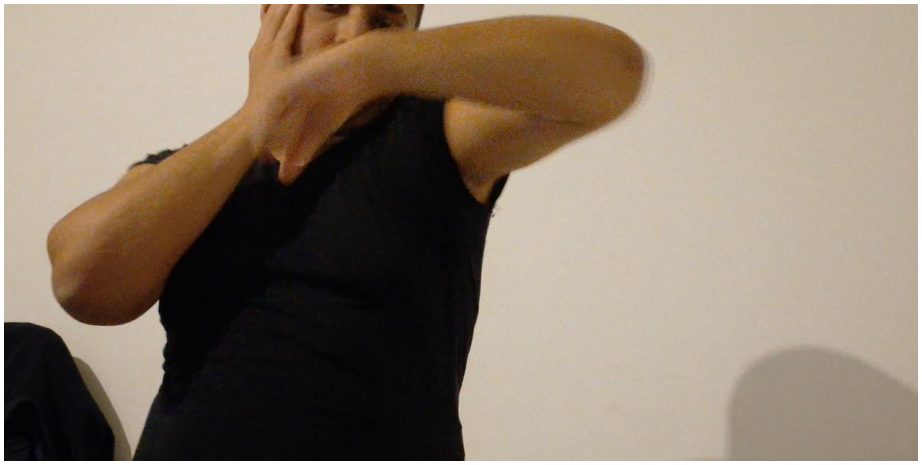
varianciával. Mindegyik ütést 200 alkalommal hajtottam végre, 100-100-as eloszlással karonként. Így összesen 600 ütés került betanításra.



Példa az egyenesre a left_straight_100 tanító videóból.



Példa a felütésre a right_uppercut_100 tanító videóból.



Példa a horogra a left_hook_100 tanító videóból.

Emellett fontos eleme volt a neutrális pozíció, mely köztes állapotot adott az ütések között. Az általános alapállásról és kisebb mozdulatokról, mint az ütem tartása, szökkenések és hajolások, két perces felvétel készült.

Példa az alapállásra a stance tanító videóból.

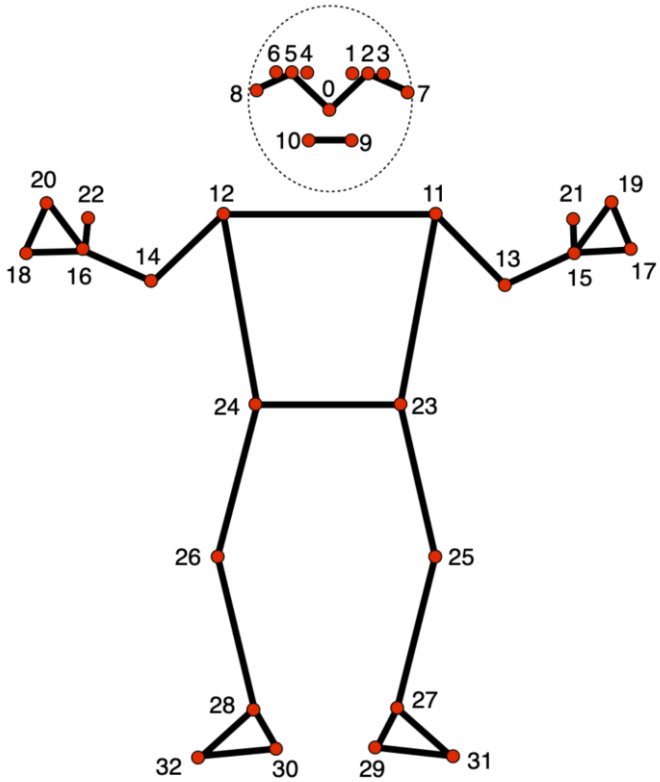
Végül, egy teszt felvételként szolgáló video készült, melyben minden elemet többször végrehajtok, ideális tesztelési környezet érdekében.

Példa képkocka a test tanító videóból.

A felvételek az alábbi linken elérhetőek: https://drive.google.com/drive/folders/154QplKnH92xWVUNlHuOEE1AraWqzXFWO?usp=drive_link

4. Koordináta felvétel

Az egyik legfontosabb eleme a programnak a Mediapipe könyvtár, ami képes emberi testek felismerésére és a test különböző pontjaira koordináta pontokat ad meg. Habár vannak kifejezetten külön testrészekre irányuló, pontosabb modelljei, ebben az esetben a teljes test felmérésére van szükségünk, így a pose_landmarker_full.task modellel dolgoztam, mely összes 33 pontot ad a detektált testnek.



A program képkockánként kielemzi a megadott videót, és az összes pont adott koordinátáját elmenti egy CSV fájlba. Emellé társítja a pontok láthatóságát, a képkockák indexét, és az ütés fajtáját is. Ezt mind az extract_coord nevű metódussal valósítja meg, melyet mindegyik videon, valamint a teszt videon is meghívunk.

```
extract_coord( video: "videok/stance.mp4",  name: "stance",  output: "data/box_koordinatak.csv")
extract_coord( video: "videok/left_straight_100.mp4",  name: "straight",  output: "data/box_koordinatak.csv")
extract_coord( video: "videok/right_straight_100.mp4",  name: "straight",  output: "data/box_koordinatak.csv")
extract_coord( video: "videok/left_hook_100.mp4",  name: "hook",  output: "data/box_koordinatak.csv")
extract_coord( video: "videok/right_hook_100.mp4",  name: "hook",  output: "data/box_koordinatak.csv")
extract_coord( video: "videok/left_uppercut_100.mp4",  name: "uppercut",  output: "data/box_koordinatak.csv")
extract_coord( video: "videok/right_uppercut_100.mp4",  name: "uppercut",  output: "data/box_koordinatak.csv")
```

	A	B	C	D	E	F	G	H	I	J	K	L	M	N	O	P	Q	R	S	T	U	V	W
1	class	frame_id	x0	y0	z0	v0	x1	y1	z1	v1	x2	y2	z2	v2	x3	y3	z3	v3	x4	y4	z4	v4	x5
2	stance	0	0.2725535	0.5841479	-1.366743	0.9922865	0.2985551	0.5005410	-1.383198	0.9914686	0.3124994	0.4930225	-1.383160	0.9902626	0.3264779	0.4846896	-1.383598	0.9930295	0.2431722	0.5092849	-1.377374	0.9872692	0.2278146
3	stance	1	0.2724948	0.5833107	-1.412840	0.9930060	0.2985611	0.5005204	-1.433723	0.9922380	0.3127132	0.4930457	-1.433558	0.9911220	0.3264497	0.4850964	-1.433907	0.9936630	0.2431014	0.5092278	-1.431034	0.9884153	0.2279143
4	stance	2	0.2683180	0.5840823	-1.440777	0.9936907	0.2955991	0.5036714	-1.471570	0.9929898	0.3112673	0.4967793	-1.471090	0.9919743	0.3245649	0.4902588	-1.471303	0.9942784	0.2389819	0.5103184	-1.464418	0.9895396	0.2252502
5	stance	3	0.2662623	0.5893931	-1.259009	0.9942908	0.2936079	0.5070112	-1.301413	0.9936411	0.3095806	0.4992494	-1.300789	0.9927067	0.3224831	0.4923871	-1.301002	0.9948096	0.2368878	0.5192375	-1.290446	0.9905221	0.2236419
6	stance	4	0.2643956	0.5963494	-1.335959	0.9948523	0.2924603	0.5098876	-1.391307	0.9942618	0.3081355	0.5013001	-1.390528	0.9934154	0.3207049	0.4940331	-1.390774	0.9953157	0.2362541	0.5276644	-1.377921	0.9914507	0.2228054
7	stance	5	0.2601556	0.6033397	-1.288295	0.9953466	0.2880007	0.5184875	-1.337991	0.9948055	0.3046759	0.5079433	-1.337300	0.9940334	0.3174505	0.4993677	-1.337623	0.9957604	0.2313494	0.5377698	-1.320762	0.9922662	0.2184344
8	stance	6	0.2587424	0.6036742	-1.260364	0.9958014	0.2873492	0.5183953	-1.303455	0.9953067	0.3037763	0.5080248	-1.302819	0.9946058	0.3163135	0.4995049	-1.303131	0.9961674	0.2305331	0.5371578	-1.290763	0.9930204	0.2171811
9	stance	7	0.2585622	0.6033881	-1.258100	0.9962103	0.2855277	0.5180506	-1.302047	0.9957575	0.3007505	0.5075852	-1.301462	0.9951197	0.3128967	0.4985777	-1.301791	0.9965308	0.2302026	0.5361677	-1.285915	0.9937007	0.2166324
10	stance	8	0.2585588	0.5962109	-1.249377	0.9965650	0.2852107	0.5098304	-1.288857	0.9961360	0.3004237	0.4994037	-1.288311	0.9955527	0.3125401	0.4902743	-1.288621	0.9968278	0.2303145	0.5275856	-1.274516	0.9942924	0.2165909
11	stance	9	0.2586025	0.5820727	-1.350977	0.9968782	0.2842109	0.4943496	-1.391395	0.9964650	0.2997377	0.4840041	-1.390841	0.9959167	0.3117254	0.4755036	-1.391226	0.9970805	0.2300883	0.5117837	-1.377323	0.9948205	0.2163798
12	stance	10	0.2587037	0.5536425	-1.385056	0.9971712	0.2838667	0.4748116	-1.413253	0.9967747	0.2993896	0.4651211	-1.412934	0.9962736	0.3111962	0.4569993	-1.413388	0.9973243	0.2302139	0.4895716	-1.400065	0.9953017	0.2164793
13	stance	11	0.2570544	0.5284130	-1.505120	0.9974207	0.2805012	0.4540438	-1.522838	0.9970247	0.2965067	0.4459547	-1.522626	0.9965687	0.3085354	0.4376057	-1.523120	0.9975129	0.2296125	0.4634712	-1.510772	0.9957093	0.2163685
14	stance	12	0.2571269	0.4922316	-1.586947	0.9976357	0.2802587	0.4178856	-1.589274	0.9972261	0.2957623	0.4103331	-1.589251	0.9968190	0.3074366	0.4029912	-1.589826	0.9976504	0.2296526	0.4257441	-1.583998	0.9960687	0.2164654
15	stance	13	0.2580908	0.4481526	-1.582856	0.9978662	0.2804349	0.3749736	-1.574837	0.9974925	0.2956180	0.3680598	-1.574703	0.9971231	0.3070393	0.3621107	-1.575153	0.9978710	0.2310415	0.3859681	-1.576102	0.9964544	0.2170148
16	stance	14	0.2588110	0.4168582	-1.511891	0.9980657	0.2802996	0.3482351	-1.532930	0.9977174	0.2945351	0.3424096	-1.532446	0.9973758	0.3054925	0.3374527	-1.532838	0.9980492	0.2332584	0.3573335	-1.520688	0.9967929	0.2188551
17	stance	15	0.2588473	0.3895007	-1.603854	0.9982126	0.2796148	0.3212179	-1.584613	0.9978621	0.2928546	0.3159942	-1.584684	0.9975512	0.3033236	0.3116153	-1.585142	0.9981439	0.2334332	0.3329989	-1.584838	0.9970628	0.2189241
18	stance	16	0.2588151	0.3650581	-1.499639	0.9983420	0.2780902	0.2999047	-1.461512	0.9980050	0.2903543	0.2957115	-1.461741	0.9977166	0.3006920	0.2917432	-1.462220	0.9982464	0.2338326	0.3093725	-1.467465	0.9973102	0.2195060

5. Adatok feldolgozása

A nyers koordináták nem megfelelőek a kitanításra. A koordináták pozícióját rengeteg tényező tudja befolyásolni, mint a test relatív pozíciója a kamerához, az adott egyén testi jellemzői és bármilyen más vizuális környezeti befolyás. Emiatt át kell dolgozni a

koordinátákat. Az angles.py-ban található vectorise nevezetű függvény, első sorban vektorizálja az eddig felvett adatokat, feldolgozási sebesség optimalizálás érdekében. Ehhez a NumPy könyvtárt használja fel. Az kapott adattömböt vektori formába konvertálja, majd a műveleteket a teljes tömbön elvégzi egyszerre, megspórolva a feldolgozási időt. Ezt követően, a vektorizált adatokat felhasználja a koszinusztétel alkalmazásával arra, hogy a test pontok közötti szögeket kiszámítsa.

```
def vectorise(A, B, C):  4 usages

    # A = vall, B = konyok, C = csuklo

    a = A.to_numpy()
    b = B.to_numpy()
    c = C.to_numpy()

    ba = a - b
    bc = c - b
    #vektorhossz
    norm_ba = np.linalg.norm(ba, axis=1)
    norm_bc = np.linalg.norm(bc, axis=1)

    # skalaris
    dot = np.sum(ba * bc, axis=1)

    # cos tetel
    cosine = dot / (norm_ba * norm_bc)

    cosine = np.clip(cosine, -1.0,  a_max: 1.0)

    return np.degrees(np.arccos(cosine))
```

A függvényt négy szög kiszámítására használjuk fel. Felvesszük a bal és jobb váll, könyök, csukló és csípő pontok összes koordinátáját, amiket a vectorise függvénnyel, felhasználjuk a könyökök és vállak pontos szögének megadására.

```
l_shoulder = coord(11)
r_shoulder =  coord(12)
l_elbow = coord(13)
r_elbow =  coord(14)
l_wrist = coord(15)
r_wrist = coord(16)
l_hip = coord(23)
r_hip = coord(24)


df['l_elbow_angle'] = vectorise(l_shoulder, l_elbow, l_wrist)
df['r_elbow_angle'] = vectorise(r_shoulder, r_elbow, r_wrist)
df['l_shoulder_angle'] = vectorise(l_hip, l_shoulder, l_elbow)
df['r_shoulder_angle'] = vectorise(r_hip, r_shoulder, r_elbow)
```

6. Modell kitanítása

A training.py-ban hajtódik végre a valódi gépi tanulási procedúra. Az adatokat két tömbre bontjuk, melyeknek egyik fele egy X tömbben tárolja magát a nyers adatokat, míg az Y az ütésfajták neveit tárolja, és számosítja, mivel a modell csak számokkal tud dolgozni. Az adatokat 30 képkockás ablakokba szeparáljuk, melyek 5 képkocka eltolással rendelkeznek, ezek kerülnek az X tömbbe. Az Y-ban tárolt számok végül One-Hot vektorokká konvertálódnak, melynek lényege, hogy elemszáma egyezik a típusok számával, és minden eleme nulla, kivéve egy, mely egyessel különbözteti meg az adott típust.

```
X = []
Y = []
cols = ['l_elbow_angle', 'r_elbow_angle', 'l_shoulder_angle', 'r_shoulder_angle']
print(f"Osztályok: {label_encoder.classes_}")
# ablakok
for name, group in df.groupby('class'):


    vals = group[cols].values
    label = group['class_encoded'].values[0]
    for i in range(0, len(vals) - win_size, step):
        window = vals[i: i + win_size]
        X.append(window)
        Y.append(label)

X = np.array(X)
Y = np.array(Y)
Y = to_categorical(Y, num_classes=NUM_CLASSES)
```

Az adatokat már előre feladatokra delegáljuk. Teljesen véletlenszerűen, az adatok 80%-át betanításra, míg a maradék 20%-ot aktív tanítás közbeni tesztelésre használjuk fel.

```
X_train, X_val, Y_train, Y_val = train_test_split( *arrays: X, Y, test_size=0.2, random_state=42)
```

A modell az egy dimenziós konvolúciós háló (1D CNN) struktúráját követi. A 30 képkockás elemeken egy 3 képkockás ablakot visz végig. 64 összefüggést keres az első réteg, mint a könyök szög hirtelen változása egy egyenesnél. Ezt követően megfelezzük az adatokat, a nagyobb értékűeket meghagyva, és ezt a kimenetet átadjuk a második rétegnek, ami már 128 összefüggést keres, sokkal komplexebb metodikával, az előző rétegre építve. Ezt is megfelezzük. Végül, a Flatten függvénnyel vektorizáljuk.

```
model.add(Conv1D(filters=64, kernel_size=3, activation='relu', input_shape=(win_size, len(cols))))
model.add(MaxPooling1D(pool_size=2))
model.add(Conv1D(filters=128, kernel_size=3, activation='relu'))
model.add(MaxPooling1D(pool_size=2))
model.add(Flatten())
```

Ennek kimenetét fogadja a Dense réteg, ami a döntéshozásért felel. 128 neuronnal rendelkezik, melyek felét véletlenszerűen kikapcsoljuk a Dropout metódussal, így pontosabb tanulásra kényszerítve a modellt. Megadjuk számára a típusokat, és a softmax paraméter miatt, a négy típusból a legnagyobb valószínűségűt kapjuk vissza. A tanulást adam algoritmussal optimalizáljuk.

```
model.add(Dense( units: 128, activation='relu'))
model.add(Dropout(0.5))
model.add(Dense(NUM_CLASSES, activation='softmax'))
model.compile(optimizer='adam', loss='categorical_crossentropy', metrics=['accuracy'])
model.summary()
```

Végül, ez a sor hajtja végre a valódi tanítást. Itt adjuk meg az X és Y adatok tömbjeit, és 40szer megyünk végig az összes tanítóablakon.

```
history = model.fit(X_train, Y_train, epochs=40, batch_size=32, validation_data=(X_val, Y_val))
```

7. Tesztelés

A teszteléshez, a fentiekben említett, erre dedikált tesztelési videot használtam fel. A feldolgozásához az eddig használt extract_coord és extract_angl függvényeket újra felhasználtam, és a teszt adatokat elmentettem. Ezeket alávetjük egy szűrőnek, ami megfigyel egy 30 képkockás ablakon belüli szórást, a könyök szögéhez mérve. Ha elég nagy a szórás, azaz mozgást lehet detektálni, átadjuk a modellnek. A modell csak akkor fogadja, ha legalább 85%-al felismeri a típust.

```
for i in range(0, len(angl_vals) - win_size, step):
    window = angl_vals[i: i + win_size]

    l_elbow_std = np.std(window[:, 0])
    r_elbow_std = np.std(window[:, 1])

    if l_elbow_std > 5.0 or r_elbow_std > 5.0:
        input_data = np.expand_dims(window, axis=0)
        prediction = model.predict(input_data, verbose=0)
        class_idx = np.argmax(prediction)
        confidence = prediction[0][class_idx]

        if confidence > 0.85:
            fps = 30
            frame_id = df['frame_id'].values[i + win_size]
            ido_mp = frame_id / fps
            matches.append((ido_mp, frame_id, classes[class_idx], confidence))
```

Végül, a modelltől visszakapott adatokat kiíratjuk egy txt szöveges fájlba. Itt szerepel a videón belüli pontos időpont, és a képkocka indexe, ahol a mozdulatot detektálta, valamint, a biztossági százalékot. Ezt konzolra is kiíratjuk.

```
with open(output, "w", encoding="utf-8") as f:
    f.write(f"Videó: {test_vid}\n")
    f.write("-" * 35 + "\n\n")

    for ido, frame, mozgas, conf in matches:
        sor = f"[{ido:05.2f}s] (Frame: {frame:04d}) - {mozgas.upper()} ({conf*100:.1f}%)\n"
        f.write(sor)
        print(sor, end="")

print(output)
```

8. Fejlesztési tervek

Habár a modell nagyobb mértékben pontos, a relatívan kevés adat erősen limitálja. Olyan külső tényezők, mint a fényhatások, adott pozíció, véletlen hibák, mind befolyásolják az eredményt. Ezeket mind orvosolni és kiegészíteni kell, annak érdekében, hogy a program a következő fázisába lépjen, az aktív kritikára.

A további fejlesztés érdekében érdemes több szögből, több kamerával készíteni a felvételeket, így még hiányos adatokat pótolva is képes a program lépést tartani. Esetleges prediktív algoritmusok segíthetik.

Jelenlegi állapotában a program minimálisan lett tesztelve, csakis a fent megadott paraméterekkel. Érdemes kísérletezni más paraméterekkel, kisebb és nagyobb ablakokkal és különböző tesztelési felvételekkel.